

Git простими словами

Урок 2 — конспект для тих, хто бачить Git уперше

1. Уяви, що ти граєш у гру

Ти проходиш складний рівень. Перед боєм з босом ти **зберігаєшся**. Якщо бос переміг — не починаєш гру з початку, а завантажуєш збереження.

Git — це збереження для коду. Ти пишеш програму, зберігаєшся, пишеш далі. Зламав усе? Завантажуєш попереднє збереження, і код знову працює, як раніше.

Збереження в Git називається «коміт» (commit).

Коміт — це фотографія всього проєкту в певний момент: усі файли, як вони виглядали, плюс твій підпис, дата і записка «що я тут зробив».

Без Git люди робили так: проєкт , проєкт_новий , проєкт_фінал , проєкт_фінал_2_точно . За тиждень уже ніхто не пам'ятає, де остання версія. З Git усе це — одна папка й акуратний список збережень.

2. Git, GitHub і GitLab — три різні речі

Це найчастіша плутанина на початку, тому розберемо одразу.

Хто це	Простими словами	Де живе
Git	програма, яка робить збереження коду	у тебе на комп'ютері
GitHub	сайт, куди можна відправити копію своїх збережень	в інтернеті
GitLab	такий самий сайт, тільки інша фірма (заснували українці)	в інтернеті або на своєму сервері

Аналогія: Git — це **двигун**. GitHub — це **гараж**, де цей двигун стоїть і де збирається вся команда механіків.

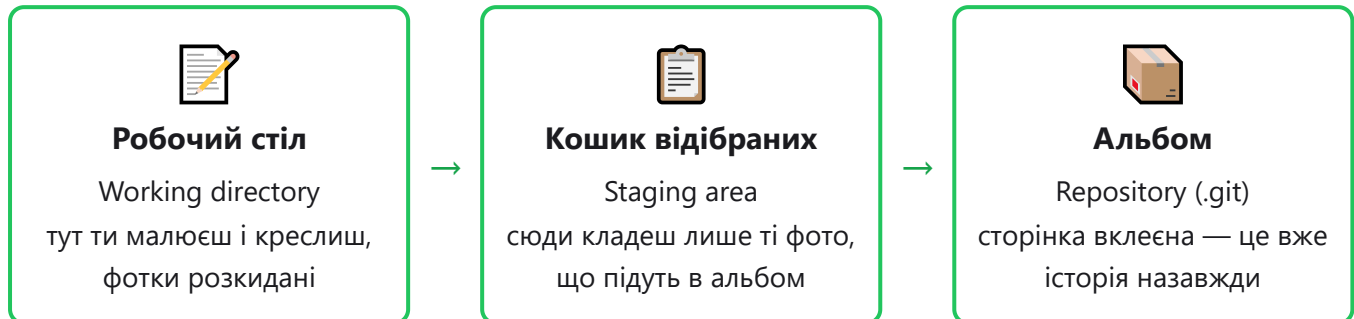
Головне: Git працює **без інтернету**. Ти можеш робити коміти в літаку. GitHub потрібен лише тоді, коли хочеш показати код іншим людям або мати копію «в хмарі», якщо з ноутбуком щось трапиться.

Звідки взявся Git

У 2005 році Лінус Торвальдс (той, хто створив Linux) розсердився, що програма для збереження версій стала платною, і написав свою — **за два тижні**. Сьогодні нею користується практично весь світ розробки.

3. Три зони: найважливіша схема уроку

Файл на шляху до «збереження» проходить три місця. Уяви, що ти збираєш фотоальбом.



А команди — це просто «перенеси з однієї зони в наступну»:

```
правлю файл  → git add  → git commit  → git push
робочий стіл   кошик    альбом      GitHub
```

Навіщо потрібен кошик (staging), чому не одразу в альбом?

Бо ти рідко робиш одну справу за раз.

Ти виправив помилку в калькуляторі і заодно змінив колір кнопки. Це дві різні історії. Через `git add` ти кладеш у коміт лише файли про калькулятор, зберігаєш його з підписом «виправив ділення», а потім окремо зберігаєш кнопку. Через місяць буде зрозуміло, що коли робилось.

4. Перші п'ять команд, які роблять усе

```
git status      # що зараз відбувається?
git add main.py # поклади файл у кошик
git commit -m "add main" # вклей сторінку в альбом
git log --oneline # покажи список збережень
git diff        # що саме я змінив?
```

Найголовніша команда — `git status`. Вона нічого не ламає, її можна писати хоч сто разів на день. Git сам пише, де ти є, що змінилось і що зробити далі. Не вгадуй — спитай `git status`.

Як виглядає перший день з Git

```
mkdir my_project          # зробили папку
cd my_project             # зайшли в неї
git init                  # «Git, стеж за цією папкою»

echo "print('Hello Git!')" > main.py

git add main.py
git commit -m "add main.py"
git log --oneline          # є перше збереження!
```

Після `git init` у папці з'являється прихована тека `.git` — це і є той самий альбом. Усередині лежить уся історія. Руками там нічого не чіпаємо, а якщо видалити цю теку — історія зникне назавжди.

5. Гілки — паралельні світи проєкту

Уяви книгу-квест: «якщо йдеш у ліс — сторінка 40, якщо в село — сторінка 60». Гілка (branch) — це саме такий поворот сюжету для коду.

```
main          A---B---C-----F    ← головна, завжди працює
               \                   /
feature-login  D---E-----        ← тут я роблю вхід у гру
```

Правило команди: у `main` **лежить те, що точно працює**. Усе нове й недороблене — в окремій гілці. Зламав щось у гілці? Головна версія навіть не помітила.

```
git checkout -b feature-login  # створити гілку і перейти в неї
git branch                    # де я? зірочка * біля поточної

# ... пишу код ...
git add .
git commit -m "add login"

git checkout main              # повернувся в головну
git merge feature-login        # влив свою роботу в головну
```

6. GitHub: віддати код у хмару

Три слова, які треба знати:

Слово	Що означає
<code>origin</code>	прізвисько для адреси твого репозиторію на GitHub
<code>git push</code>	«штовхнути» — відправити свої коміти в хмару
<code>git pull</code>	«потягнути» — забрати те, що додали інші

```
git remote add origin git@github.com:username/my_project.git
git push -u origin main    # перший раз — з -u
git push                   # далі просто так
git pull                   # забрати чужі зміни
```

Головна звичка: починай роботу з `git pull`, а закінчуй `git push`. Тоді ти завжди працюєш зі свіжою версією, і твоя робота не лежить лише на одному ноутбучі.

SSH-ключ — це твій підпис, а не пароль

Щоб GitHub впускав тебе без пароля, роблять пару ключів командою `ssh-keygen`.

З'являються два файли:

- `id_rsa` — **приватний** ключ. Це як ключ від квартири: нікому, ніколи, ні в яких чатах.
- `id_rsa.pub` — **публічний** ключ. Це як замок: його спокійно віддаємо GitHub (Settings → SSH and GPG keys).

GitHub чіпляє твій «замок» собі, і далі твій ключ його відкриває сам, без жодних паролів.

7. Pull Request — «перевір, будь ласка, мою роботу»

У командах не заливають зміни в головну гілку напряму. Спочатку кажуть: «ось що я зробив, подивіться». Це і є **Pull Request** (на GitLab його називають **Merge Request**).

1. запусив свою гілку на GitHub;
2. натиснув **Compare & pull request**;
3. написав, що змінив і як це перевірити;
4. додав рев'юера — людину, яка прочитає код;
5. вона коментує, ти виправляєш;
6. після схвалення тиснемо **Merge** — зміни в головній гілці.

Це не бюрократія: свіже око знаходить помилки, які автор перестав бачити після третьої години роботи.

8. Конфлікт — двоє писали в одному рядку

Ти змінив рядок 10, і колега змінив рядок 10. Git не вміє вибирати, чия версія краща, тому чесно показує обидві й просить розібратися:

```
<<<<<< HEAD
print("Hello world")      ← моя версія
=====
print("Hello Git!")       ← версія з іншої гілки
>>>>>> feature-branch
```

Що робимо: відкриваємо файл, залишаємо правильний рядок, видаляємо дивні значки `<<<`, `===`, `>>>`. Потім:

```
git add file.py
git commit -m "resolve conflict"
```

Конфлікт — це не поломка. Це нормальна робоча ситуація, вона буває в усіх. Git навіть підстрахує: `git merge --abort` скасує злиття й поверне все, як було до нього.

9. Кишення на швидку руку: `git stash`

Ти щось недороблене пишеш, і раптом треба терміново перейти в іншу гілку. Комітити напівроботу не хочеться. Складаємо в кишеню:

```
git stash      # відклав, робоча папка чиста
git stash list # що в кишені лежить
git stash pop  # повернув назад
```

10. Що робити, коли «я все зламав»

Ситуація	Команда
наплутав у файлі, коміту ще не було	<code>git restore file.py</code>
додав у кошик зайвий файл	<code>git restore --staged file.py</code>
хочу скасувати вже зроблений коміт	<code>git revert <хеш></code>
хочу подивитись, як було раніше	<code>git checkout <хеш></code>
здається, я втратив свої коміти	<code>git reflog</code>

Добра новина: у Git майже неможливо втратити роботу назавжди. Якщо коміт колись існував, Git тримає його ще близько 30 днів — і `git reflog` покаже, куди повернутись.

Головне з уроку — п'ять рядків

1. **Коміт** — це збереження в грі. Робіть їх часто й з підписами по суті.
2. **Три зони:** файл → `git add` → `git commit` → `git push`.
3. **Git ≠ GitHub.** Git у тебе, GitHub — гараж у хмарі.
4. **Гілка** — окремий світ для нової ідеї, щоб не зламати робочу версію.
5. **Не знаєш, що робити** — пиши `git status`.